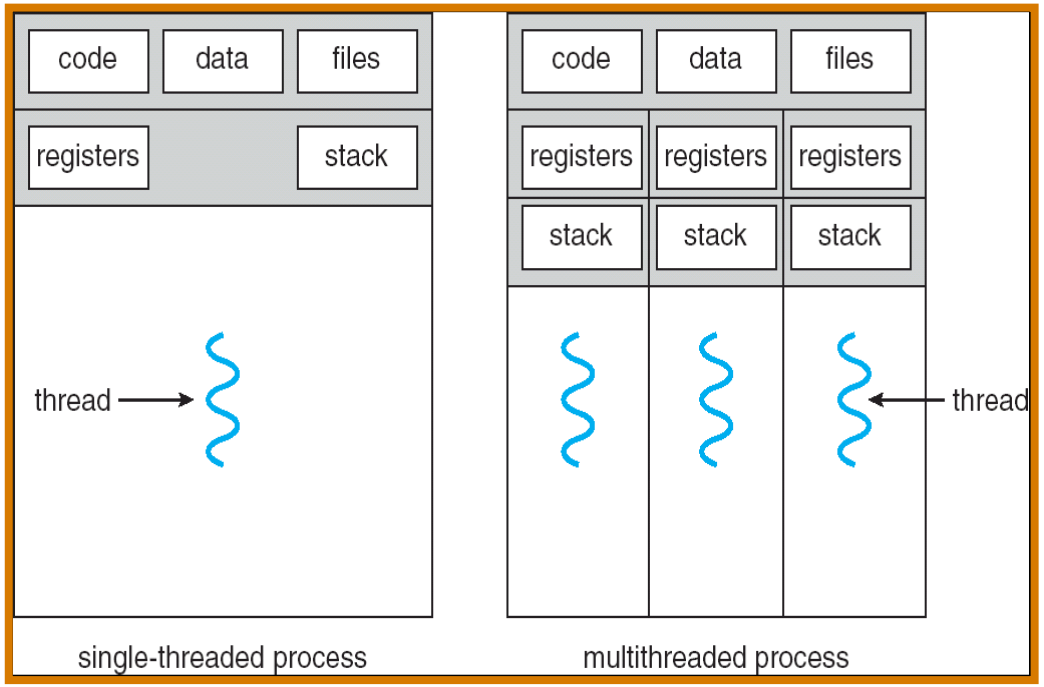
Threads

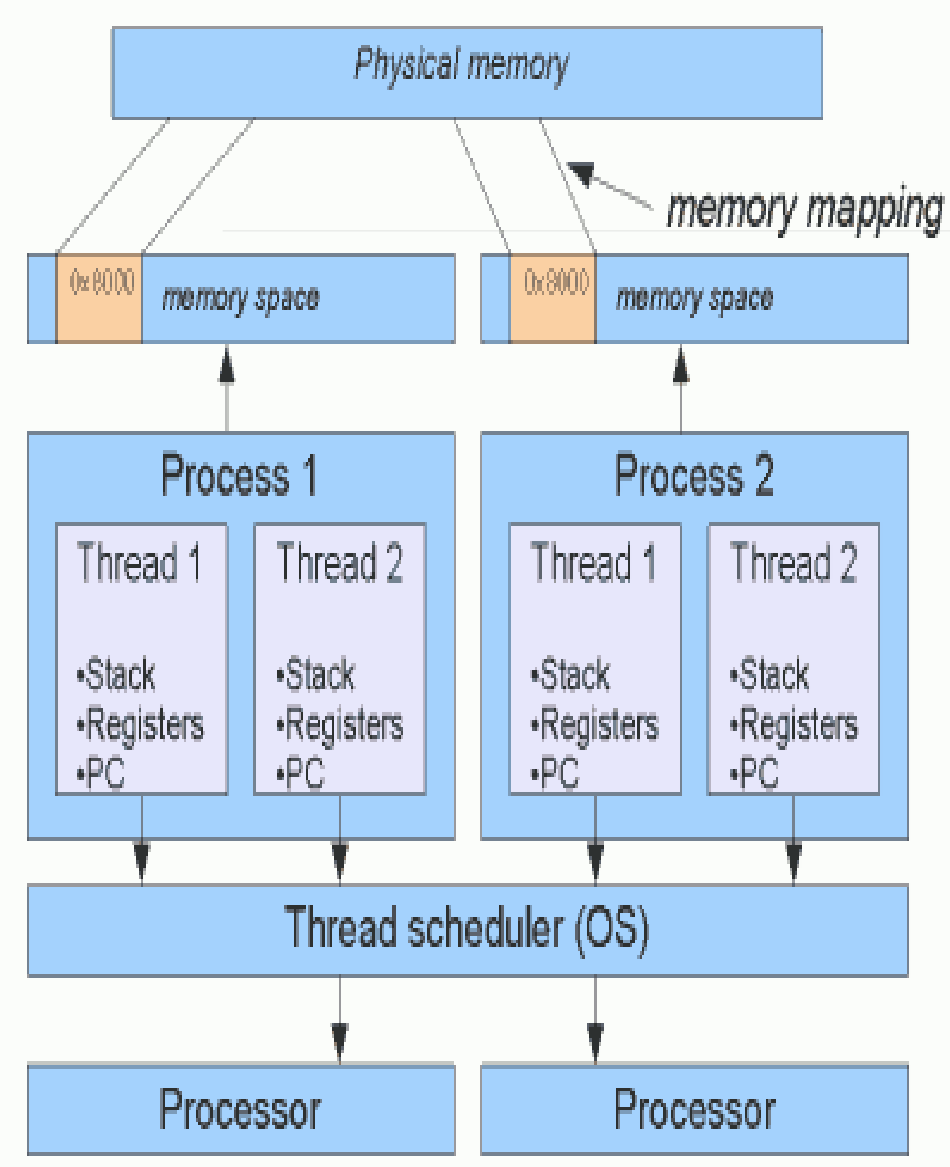
Outline

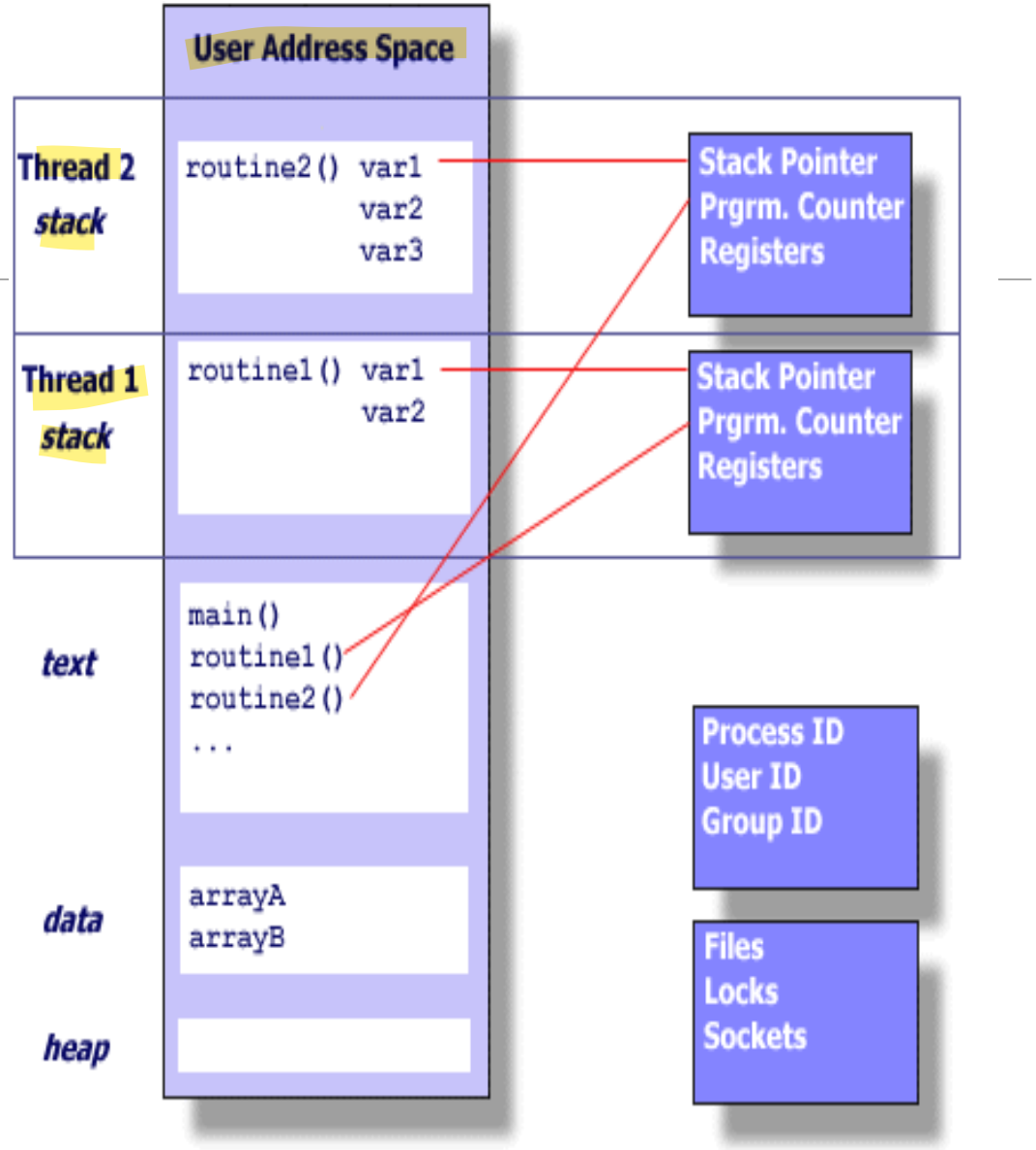
- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues
- Operating System Examples

Threads Overview

- Multithreading: a single program made up of a number of different concurrent activities
- A thread (or lightweight process)
  - basic unit of CPU utilization; it consists of: program counter, register set and stack space
  - A thread shares the following with peer threads:
    - code section, data section and OS resources (open files, signals)
    - No protection between threads




Physical memory
memory mapping
0x8000
memory space
0x8000
memory space
Process 1
Thread 1
•Stack
•Registers
•PC
Thread 2
•Stack
•Registers
•PC
Process 2
Thread 1
•Stack
•Registers
•PC
Thread 2
•Stack
•Registers
•PC
Thread scheduler (OS)
Processor
Processor


User Address Space
Thread 2
stack
routine2() var1
var2
var3
Thread 1
stack
routine1() var1
var2
Stack Pointer
Prgrm. Counter
Registers
Stack Pointer
Prgrm. Counter
Registers
text
main()
routine1()
routine2()
...
data
arrayA
arrayB
heap
Process ID
User ID
Group ID
Files
Locks
Sockets

Benefits

Responsiveness – may allow continued execution if part of process is blocked, especially important for user interfaces

Resource Sharing – threads share resources of process, easier than shared memory or message passing

Economy – cheaper than process creation, thread switching lower overhead than context switching

Scalability – process can take advantage of multicore architectures

Challenges

Multicore or **multiprocessor** systems puts pressure on programmers, challenges include:

- **Dividing activities**
- **Balance**
- **Data splitting**
- **Data dependency**
- **Testing and debugging**

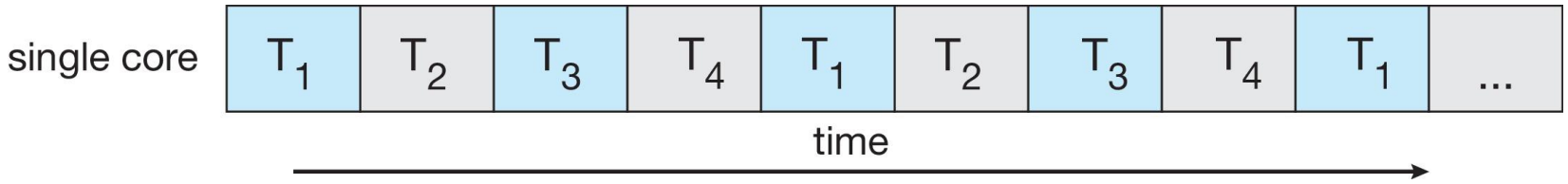
Parallelism implies a system can perform more than one task simultaneously

Concurrency supports more than one task making progress

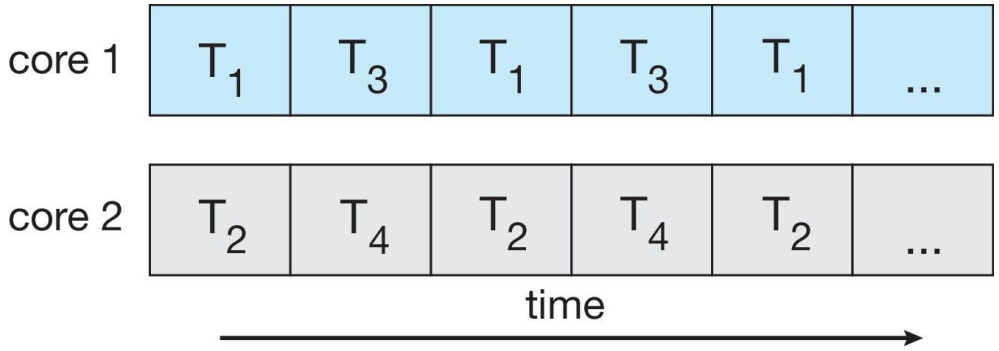
- Single processor / core, scheduler providing concurrency

Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



- **Parallelism on a multi-core system:**

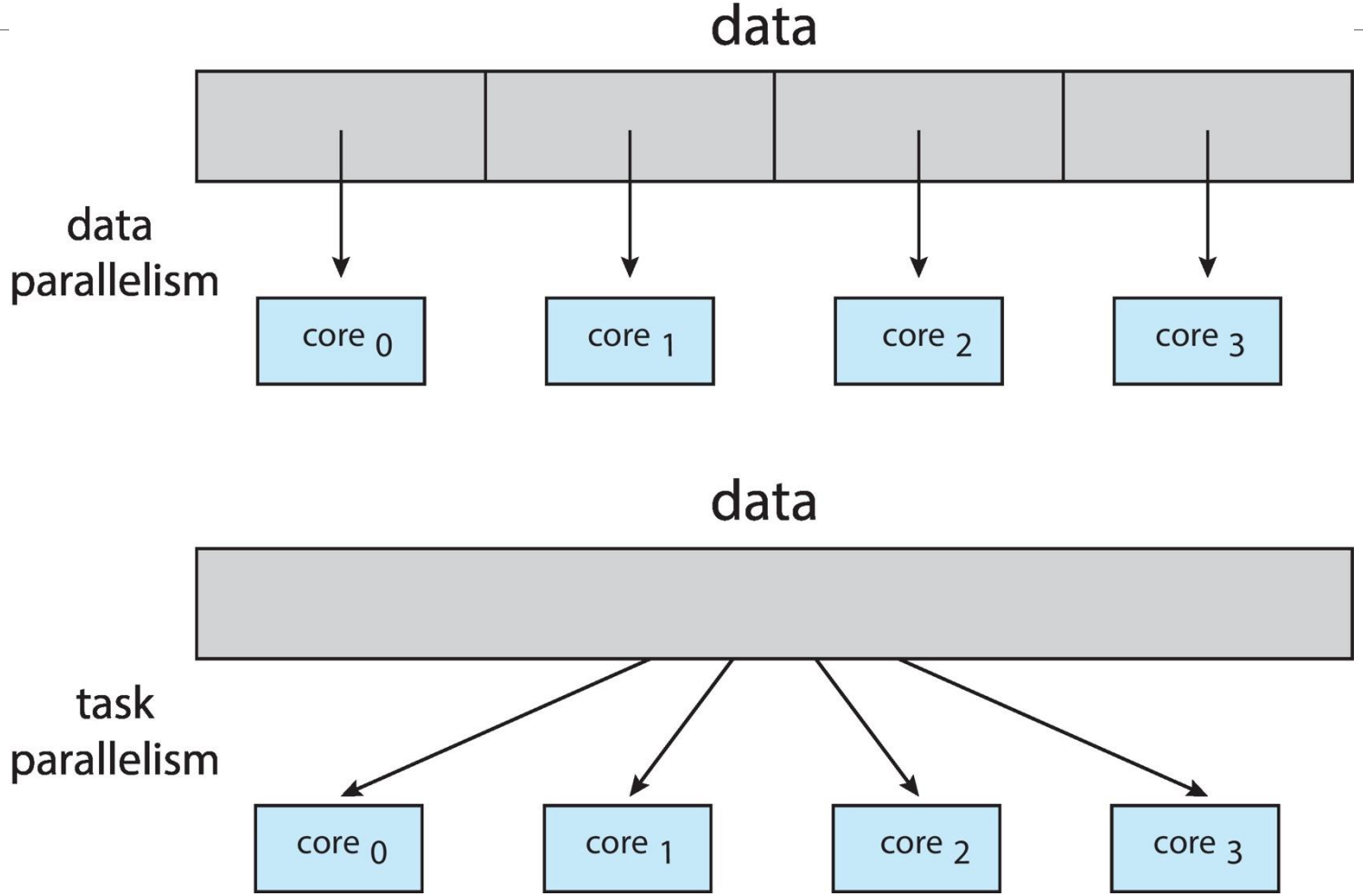


Multicore Programming

Types of parallelism

- **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
- **Task parallelism** – distributing threads across cores, each thread performing unique operation

Data and Task Parallelism



Amdahl's Law

Identifies performance gains from adding additional cores to an application that has both serial and parallel components

S is serial portion

N processing cores

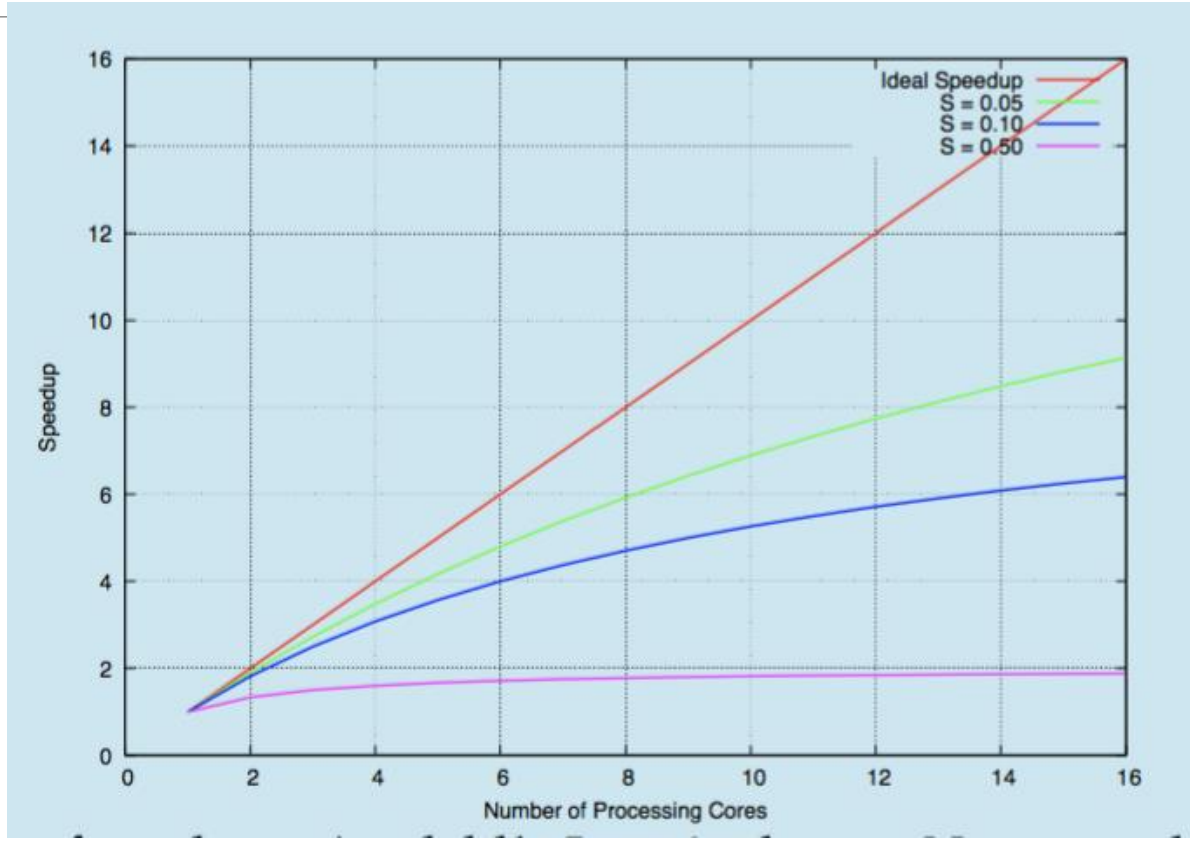
$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times

As *N* approaches infinity, speedup approaches 1 / *S*
Serial portion of an application has disproportionate effect on performance gained by adding additional cores

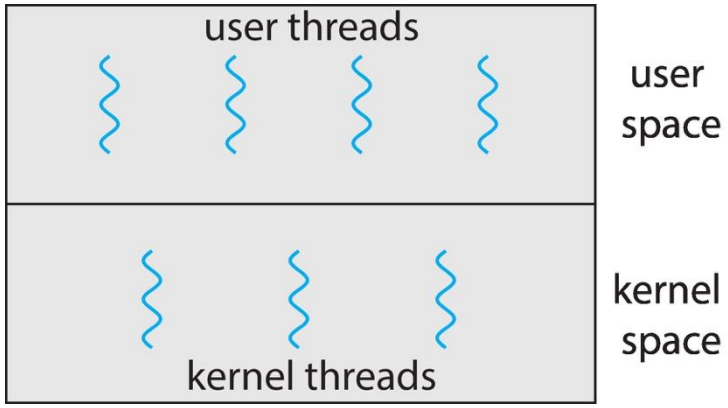
But does the law take into account contemporary multicore systems?

Amdahl's Law…


Ideal Speedup
S = 0.05
S = 0.10
S = 0.50
Speedup
Number of Processing Cores
0
2
4
6
8
10
12
14
16

Threads Overview…

- Types Of Threads
  - Kernel Threads: Supported by the Kernel
    - Kernel performs thread creation, scheduling and management in kernel space.
    - Expensive(slower to create and manage)
    - Kernel can schedule threads on different processors.
  - User Threads: management done by user-level threads library
    - Supported above the kernel, via a set of library calls at the user level.
    - Cheap, Faster than kernel threads.
    - Example Pthread in Unix.
    - If kernel is single threaded, blocking system call from any thread can block the entire task.

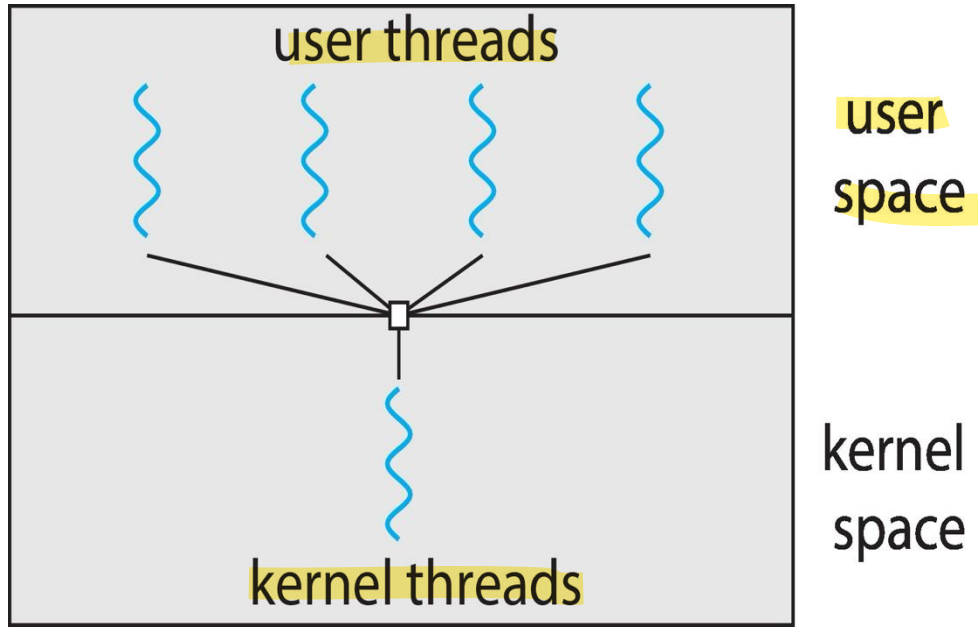


Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**



One-to-One

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Can burden the performance of an application. Creating a user thread requires creating the corresponding kernel thread. This restricts the number of user thread creation.
- Examples
  - Windows
  - Linux

Kernel threads are managed by the OS kernel, so creating them involves:

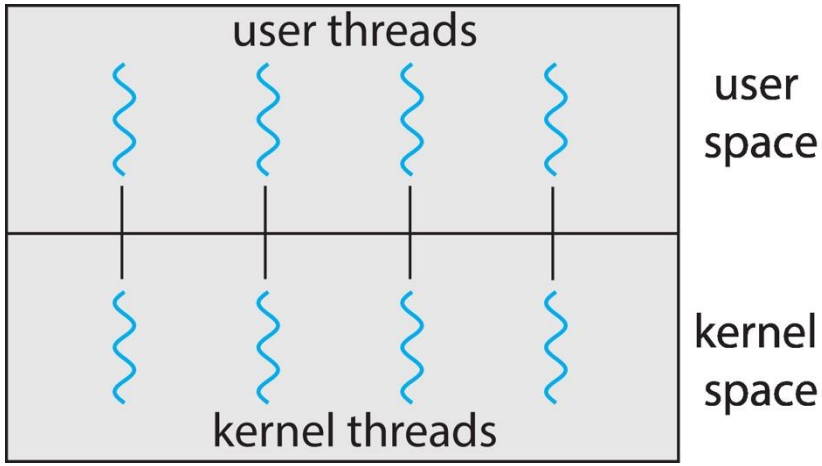
system calls

memory allocation

kernel data structures

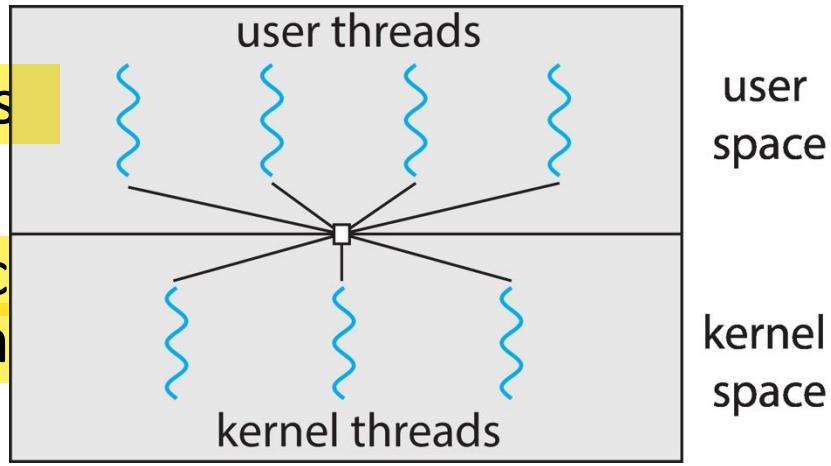
scheduling overhead

So thread creation becomes expensive.



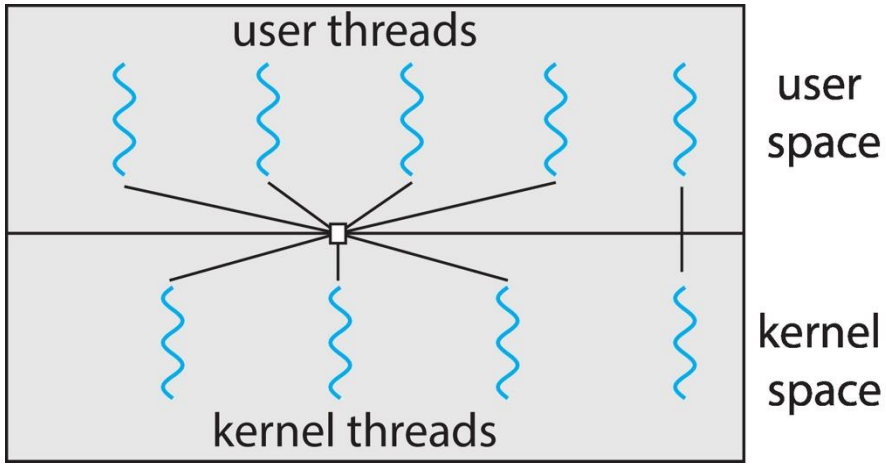
Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads . Ex: Solaris 2, Windows with the *ThreadFiber* package
- Allows the operating system to create a sufficient number of kernel threads
- The number of kernel threads may be specific to either an application or machine(ex:more kernel thread on MP and les on uniprocessor).
- When a thread performs a blocking system call , the kernel c schedule another thread for execution. Also allows to run in parallel on a MP.



Two-level Model

Similar to M:M, except that it allows a user thread to be **bound** to kernel thread



So there are two possibilities:

2

That is why it is called Two-Level.

Thread Libraries

Thread library provides programmer with API for creating and managing threads

Two primary ways of implementing

- Library entirely in user space
- Kernel-level library supported by the OS

Pthreads

May be provided either as user-level or kernel-level

A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization

Specification, not ***implementation***

API specifies behavior of the thread library, implementation is up to development of the library

Common in UNIX operating systems (Linux & Mac OS X)

Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```

Pthreads Example...

```
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```

Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
   pthread_join(workers[i], NULL);
```

Threading Issues

- Semantics of **fork()** and **exec()** system calls

Signal handling
  - Synchronous and asynchronous

- Thread cancellation of target thread
  - Asynchronous or deferred

- Thread-local storage

- Thread Pools

- Scheduler Activations

Semantics of fork() and exec()

Does **`fork()`** duplicate only the calling thread or all threads?

- Some UNIXes have two versions of fork

`exec()` usually works as normal – replace the running process including all threads

Signal Handling

Signals are used in UNIX systems to notify a process that a particular event has occurred.

A **signal handler** is used to process signals

1. Signal is generated by particular event
2. Signal is delivered to a process
3. Signal is handled by one of two signal handlers:
    1. default
    2. user-defined

Every signal has **default handler** that kernel runs when handling signal

- **User-defined signal handler** can override default
- For single-threaded, signal delivered to process

Signal Handling…

Where should a signal be delivered for multi-threaded?

- Deliver the signal to the thread to which the signal applies
- Deliver the signal to every thread in the process
- Deliver the signal to certain threads in the process
- Assign a specific thread to receive all signals for the process

Thread Cancellation

Terminating a thread before it has finished

Thread to be canceled is **target thread**

Two general approaches:

- **Asynchronous cancellation** terminates the target thread immediately
- **Deferred cancellation** allows the target thread to periodically check if it should be cancelled

Pthread code to create and cancel a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

  . . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid,NULL);
```

Thread Cancellation…

Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

If thread has cancellation disabled, cancellation remains pending until thread enables it

Default type is deferred

- Cancellation only occurs when thread reaches **cancellation point**
  - i.e., `pthread_testcancel()`
  - Then **cleanup handler** is invoked

On Linux systems, thread cancellation is handled through signals

Thread-Local Storage

Thread-local storage (**TLS**) allows each thread to have its own copy of data

Useful when you do not have control over the thread creation process (i.e., when using a thread pool)

Different from local variables

- Local variables visible only during single function invocation
- TLS visible across function invocations

Similar to **`static`** data

- TLS is unique to each thread

Operating System Examples

- Linux Threads

Linux refers to them as ***tasks*** rather than ***threads***

Thread creation is done through **`clone()`** system call

`clone()` allows a child task to share the address space of the parent task (process)

- Flags control behavior

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

`struct task_struct` points to process data structures (shared or unique)